

08 — Orchestration Service: Advanced Modules

What it is: the remaining orchestration building blocks — managed RAG, guaranteed-JSON output, multi-turn memory — plus how sheet 07's pieces combine into one runnable agent.

Document Grounding (managed RAG)

```
from gen_ai_hub.orchestration.models.document_grounding import (
    DocumentGrounding, DocumentGroundingFilter, GroundingModule)

grounding = DocumentGrounding(module_config=GroundingModule(
    input_params=["groundingRequest"],      # placeholder holding the query to search with
    output_param="groundingOutput",        # placeholder filled with retrieved context
    filters=[DocumentGroundingFilter(id="f1", data_repositories=["*"])],
))
# Template must reference BOTH placeholders:
# UserMessage("Question: {{?groundingRequest}}\nContext: {{?groundingOutput}}")
```

Compare with sheet 05: there, YOU write retrieval code. Here, AI Core retrieves + injects for you.

Structured Output (guaranteed JSON)

```
from gen_ai_hub.orchestration.models.response_format import ResponseFormatJsonSchema

SCHEMA = {"type": "object", "properties": {
    "category": {"type": "string", "enum": ["BUG", "QUESTION"]},
    "urgency": {"type": "string", "enum": ["LOW", "HIGH"]},
    "required": ["category", "urgency"], "additionalProperties": False}

template = Template(
    messages=[SystemMessage("Extract fields as JSON."), UserMessage("{{?text}}")],
    response_format=ResponseFormatJsonSchema(name="extract", schema=SCHEMA, strict=True),
)
result_dict = json.loads(result.orchestration_result.choices[0].message.content) # safe to parse
```

Conversation History (orchestration is STATELESS)

```
class ConversationBuffer:
    def __init__(self, max_turns=10):
        self.history = [] # list of Message objects

    def add_user(self, text): self.history.append(UserMessage(text)); self._trim()
    def add_assistant(self, text): self.history.append(AssistantMessage(text)); self._trim()
    def _trim(self):
        self.history = self.history[-(self.max_turns * 2):] # sliding window, oldest dropped

result = service.run(config=config, template_values=[...],
    messages_history=buffer.history) # <- YOU resend history every call
```

Full Pipeline + Agent (everything wired together)

```
def build_full_config(system_prompt, use_grounding=False, use_masking=True, use_filtering=True):
    kwargs = {"template": Template(...), "llm": build_llm()}
    if use_masking: kwargs["data_masking"] = build_masking(...)
    if use_filtering: kwargs["filtering"] = build_filtering(...)
    if use_grounding: kwargs["grounding"] = DocumentGrounding(...)
    return OrchestrationConfig(**kwargs)

class OrchestrationAgent: # class = config builder + buffer + .ask()
    def ask(self, text):
        try:
            result = self.orchestration_service.run(config=self._build_config(),
```

```

        template_values=[TemplateValue(name="text", value=text)],
        messages_history=self.buffer.history)
    answer = result.orchestration_result.choices[0].message.content
    self.buffer.add_user(text); self.buffer.add_assistant(answer)
    return answer
except Exception as e:
    return f"Request blocked or failed: {e}"      # graceful degrade, not a crash

```

Key Points

- Grounding placeholders are **linked by name**: `input_params` tells the module which placeholder to search with; `output_param` is where results get written back.
- `strict: true + additionalProperties: false` in a JSON schema guarantees the model can't emit extra/unexpected keys — safe to feed straight into business logic.
- The Orchestration Service has **no session memory** — `messages_history` must be rebuilt and resent by the client on every single call, exactly like the RAG memory in sheet 05.
- `OrchestrationAgent.ask()` catches exceptions and returns a message instead of raising — a content-filter block is an expected outcome for a chat UI, not a bug to crash on.

Interview Q&A

Q: How would you choose between orchestration grounding and a hand-rolled RAG pipeline (sheet 05)? A: Managed grounding for standard document Q&A with less code to maintain; hand-rolled RAG when you need custom retrieval logic — hybrid search, re-ranking, or joining retrieval with live SQL data.

Q: Why is strict JSON schema output more reliable than asking the model to “respond in JSON” in the prompt? A: A prompt instruction is a suggestion the model can drift from; `response_format` constrains token generation itself, so the output is structurally guaranteed to validate — no regex cleanup needed.

Q: What's the risk of NOT capping conversation history length? A: Token cost (and latency) grows every turn as the full history gets resent — a sliding window bounds cost at the expense of the model “forgetting” very old turns.